

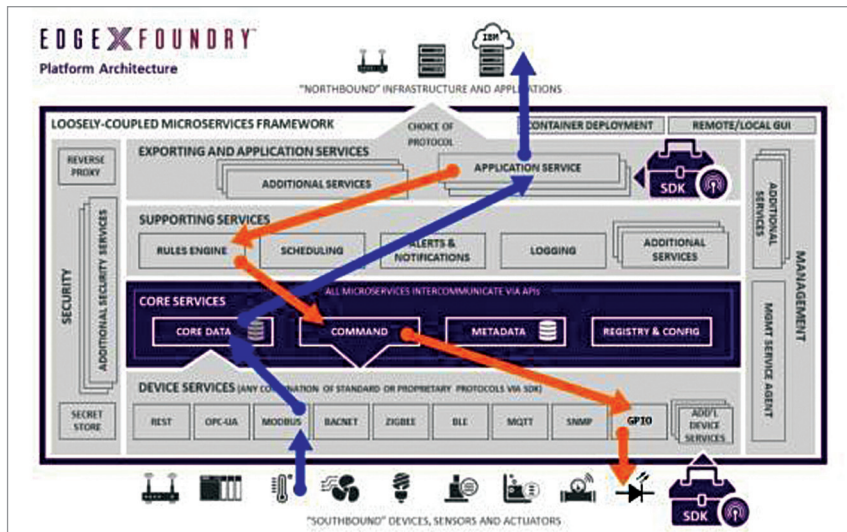


Fig. 7: EdgeX foundry

```

root@ubuntu:/home/ubuntu# curl -X "GET" http://localhost:48082/api/v1/device/3cbce3d8-c8ac-4ffb-a40f-fb9f7b5a8126/command/4ff15255-8e5b-4d3c-b5c8-22b6f136e1f1 | jq
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 267 100 267 0 0 1733 0 --:--:-- --:--:-- --:--:-- 1733
{
  "device": "TemperatureHumiditySensor",
  "origin": "1616337409901784000",
  "readings": [
    {
      "origin": "1616337409899003600",
      "device": "TemperatureHumiditySensor",
      "name": "TemperatureDeg",
      "value": "3.360000e+01",
      "valueType": "Float32",
      "floatEncoding": "eNotation"
    }
  ],
  "EncodedEvent": null
}
root@ubuntu:/home/ubuntu# curl -X "GET" http://localhost:48082/api/v1/device/3cbce3d8-c8ac-4ffb-a40f-fb9f7b5a8126/command/b2c2bed7-ef3f-47ec-a6bb-66c74fbcebe6 | jq
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 269 100 269 0 0 2537 0 --:--:-- --:--:-- --:--:-- 2537
{
  "device": "TemperatureHumiditySensor",
  "origin": "1616337414906200000",
  "readings": [
    {
      "origin": "1616337414903853000",
      "device": "TemperatureHumiditySensor",
      "name": "HumidityPercentRH",
      "value": "6.190000e+01",
      "valueType": "Float32",
      "floatEncoding": "eNotation"
    }
  ],
  "EncodedEvent": null
}

```

Fig. 8: Sensor values captured at core service

Similarly, Humidity value received is 0x021a, which is 538 in decimal. SHT20 sends the values in multiple of 10, so humidity is 53.8% RH.

Verifying LED connectivity

To check the needed functionality of breadboard connections with LEDs, a breadboard, an LED, resistors, and jumper wires are required.

Like Modbus connectivity verification, here too the objective is to verify the breadboard and LEDs connectivity without using EdgeX

in first place. The logical diagram in Fig. 4 shows connection of GPIO pins of Raspberry Pi with LEDs present on breadboard using <https://www.circuit-diagram.org/editor/>

The circuit diagram provides a convenient way of depicting the connections.

The author has tested the LEDs functionality with a Python program. Package RPi.GPIO provides a class to control the GPIO on a Raspberry Pi. Breadboard actual connections and output of green LED glow is depicted in Fig. 4, which shows

circuit connections of GPIO pins to LEDs on breadboard, while Fig. 5 shows the actual connections of GPIO pins to LEDs on breadboard.

End-to-end data flow

The objective of this case study is to use EdgeX framework to read the data from the sensor and send it to cloud to actuate LEDs whenever the temperature reaches upper or lower threshold values. Raspberry Pi is the IIoT gateway. Corresponding physical connections for the end-to-end data flow are shown in Fig. 6.

Micro services within EdgeX

Data flow indicated with blue arrows in Fig. 7 is from southbound to northbound, that is, from the sensor to cloud. Data flow indicated with orange arrows is from northbound to southbound, that is, from the application service to LED actuation.

For the data flow from southbound to northbound (sensor to cloud):

- Modbus device service retrieves the temperature and humidity values from the sensor using Modbus messages every 5 seconds. Pulling data every 5 seconds is done using Auto Events in Modbus device service.
 - Modbus device service applies scale factor of 0.1 before sending the sensor data to core services.
 - Sensor data is stored temporarily in core services and received at application service based on the configuration within application service.
 - Sensor data values can be fetched and verified using core command REST APIs.
 - Application service will forward sensor values to the cloud configured in the configuration.toml file.
 - You can monitor the sensor values using the IBM Cloud Portal.
- For the data flow from north-